Clean Code Report

Project: BookStore “Training Brain”
Authors: Team CtrlZ
Reference book: Robert C. Martin, *Clean Code: A Handbook of Agile Software Craftsmanship* (Prentice Hall, 2008)

1. Introduction

This document relates the main ideas of Robert C. Martin’s *Clean Code* to the Java console application we built (packages `model`, `controller` and `view`). For every chapter we explain the principle in our own words, show where it appears in the code, and give an honest level of compliance. Section 4 lists the places where the code does not follow the book yet.

2. Summary: Book Principles vs. Our Code

Book chapter	Principle (short)	Where it shows in our code	Level
Meaningful Names	Names reveal intent	`checkReorderStatus()`, `calculateSubtotal()`, `BookstoreData.REORDER_LEVEL`, `testSetPriceNegativeIsRejected()`	Applied
Functions	Small, do one thing, few arguments	`Main` has more than 40 small methods; `pay()` and `returnStock()` were extracted from `checkout()`	Mostly applied
Comments	Code should explain itself; comment only the why	Javadoc only on non-obvious helpers such as `menu()`, `selectBook()`, `printAll()`, `attempt()`	Partially applied
Formatting	Consistent layout, newspaper-style ordering	4-space indentation, one class per file, `main` first and helpers below, section banners in `Main`	Applied
Objects and Data Structures	Hide data, expose behavior	Private fields in all model classes; behavior inside objects (`Book.updateStock()`, `Order.addDetail()`)	Mostly applied

Book chapter	Principle (short)	Where it shows in our code	Level
Error Handling	Use exceptions with clear messages	`IllegalArgumentException` in `Customer`, `OrderDetail`, `ShoppingCart`; `attempt()` in `Main`	Partially applied
Boundaries	Isolate third-party and external code	File and JSON handling isolated in `JsonStorage` and `JsonUtil`; CSV isolated in `ReportService`	Applied
Unit Tests	Clean tests, F.I.R.S.T.	15 test classes, 166 JUnit tests, descriptive names, fresh objects per test	Applied
Classes	Small, single responsibility	One concept per model class; separate classes for storage, reports and utilities	Partially applied
Smells and Heuristics	Avoid duplication and magic numbers	Generic helpers (`Lists.find`, `Lists.nextId`, `menu`, `addNamed`, `printAll`, `readNumber`)	Mostly applied

3. Detailed Analysis

3.1. Meaningful Names

Principle: a name should tell why something exists, what it does and how it is used, so that no comment is needed to understand it.

In our code:

- Classes are nouns from the domain: `Book`, `Author`, `Publisher`, `ShoppingCart`, `DeliveryAssignment`.
- Methods are verbs that describe the action: `confirmOrder()`, `cancelOrder()`, `assignDriver()`, `completeDelivery()`, `postReview()`.
- Booleans read like questions: `checkAvailability()`, `validatePayment()`, `isRead()`.
- The reorder threshold is a named constant (`BookstoreData.REORDER_LEVEL`), not the number 3 repeated in several places.
- Test names describe the behavior being verified, for example `testPostSameReviewTwiceDoesNotDuplicate()` and `testCancelDeliveredOrderIsNotAllowed()`.

3.2. Functions

Principle: functions should be small, do one thing, work at one level of abstraction and take few arguments.

In our code:

- `main()` contains a single call to `menu(...)`. All the details live in lower-level methods.
- `checkout()` delegates payment to `pay(Order)` and stock restoration to `returnStock(Order)` instead of doing everything itself.
- `ShoppingCart.updateStock()` first validates every item and only then subtracts the stock, so a failed checkout never leaves the stock half-modified.
- Most methods in `Main` are between 5 and 25 lines. The longest ones are `checkout()` (42 lines), `addBook()` (40) and `pay()` (31).
- Input helpers such as `readInt(String)` and `readFloat(String)` take one argument and share the retry logic in `readNumber(...)`.

3.2.1. Do not repeat yourself inside functions

The four menus (main, admin, customer access, customer, reports) used to repeat the same loop. They now call one method, `menu(title, exitLabel, prompt, exitMessage, action, options...)`, which shows the options, reads the choice, validates it and saves the data.

3.3. Comments

Principle: a comment is a failure to express the idea in code. Good comments explain intent or warn about consequences; bad comments repeat the code.

In our code:

- Javadoc is used only where the name alone is not enough, for example `selectBook(String)` (“shows the books, asks for an id and returns the book or null”).
- Short inline comments explain a decision, such as the automatic saving of the JSON files after each menu action.
- Test classes use one comment per defect (`DEF-BOK-01`, `DEF-ORD-01`, ...) to explain which rule is being checked.

3.4. Formatting

Principle: code is read much more often than it is written, so layout must be consistent and follow a top-down order (the “newspaper” metaphor).

In our code:

- One public class per file, consistent 4-space indentation and braces on every `if` and `for`.
- In `Main`, the entry point comes first, then the menus, then the admin and customer actions, and finally the shared helpers. Banner comments separate each group.
- Related methods are close together (`checkout()`, `pay()`, `returnStock()`), and lines stay within a readable length.

3.5. Objects and Data Structures

Principle: objects hide their data and expose behavior. Data structures expose data and have little behavior.

In our code:

- Every model class keeps its fields `private` and exposes them through getters and setters, so each rule is checked in one place (`Customer.setAge()` accepts only 18 to 100, `Customer.setPassword()` requires 8 characters).
- Behavior lives with the data: `Book.updateStock()` and `Book.getInfo()`, `Order.addDetail()` (which recalculates the total), `ShoppingCart.calculateSubtotal()`.
- The view does not touch the internals of the model; it asks objects to do the work.

3.6. Error Handling

Principle: use exceptions instead of error codes, give them clear messages, and do not let them hide the main logic.

In our code:

- Invalid data raises `IllegalArgumentException` with a message that explains the problem (“The password must have at least 8 characters”, “The quantity must be greater than 0”, “Not enough stock for: <title>”).
- `Main.attempt(Runnable)` is the single place where validation errors are caught and shown to the user as “Error: …”, so the business flow in `addBook()` or `registerCustomer()` stays free of `try/catch` noise.
- `requireText(value, label)` centralizes the “cannot be empty” check.

3.7. Boundaries

Principle: keep the code that talks to the outside world (files, formats, libraries) separate from the business logic.

In our code:

- Reading and writing the JSON data files is done only by `JsonStorage`, with the parsing in `JsonUtil`. The model classes know nothing about JSON.
- CSV reports are generated only by `ReportService`; `Main` just calls it and prints the saved path.
- Keyboard input goes through `readLine`, which also handles an unexpected end of input, so the rest of the code never touches `Scanner` directly.

3.8. Unit Tests

Principle: tests are as important as production code. Clean tests are readable and follow F.I.R.S.T.: Fast, Independent, Repeatable, Self-validating, Timely.

In our code:

- **Fast:** the 166 tests run in about three seconds.
- **Independent:** most test classes build fresh objects in `setUp()`, so the order of execution does not matter.
- **Repeatable:** tests use only in-memory objects, with no files, network or database.
- **Self-validating:** every test ends in an assertion or an expected exception; the result is pass or fail, never a manual check.
- **Timely (partial):** every model class has its own test class, but the controller classes (`JsonStorage`, `ReportService`) are not tested yet.
- One concept per test and a descriptive name, for example `testSetEmailInvalidFormatIsRejected()`.
- Tests were written against the expected business rules, so they found 16 real defects instead of only confirming the current behavior.

3.9. Classes

Principle: classes should be small and have a single responsibility, with high cohesion.

In our code:

- The 15 model classes each represent one domain concept.
- Responsibilities are split by package: `model` (domain), `controller` (storage, reports and utilities) and `view` (console interface).
- `JsonStorage` only persists, `JsonUtil` only parses, `ReportService` only writes CSV, and `Lists` only offers generic list helpers.

3.10. Smells and Heuristics

Principle: remove duplication, avoid magic numbers, and prefer clear code over clever code.

In our code:

- **Duplication removed:** `addNamed(...)` is shared by “add author”, “add publisher” and “add category”; `printAll(...)` is shared by every listing; `Lists.find(...)` and `Lists.nextId(...)` work for any entity type; the “generate all reports” option reuses the single-report code.
- **Named constant:** `BookstoreData.REORDER_LEVEL`.
- **Dead code:** unused imports and empty blocks were removed from `Main`.

4. Areas for Improvement

Book principle	What we found	Suggested improvement
Comments	Getters and setters in the model have auto-generated Javadoc (`@return the id`) that repeats the method name	Delete the redundant Javadoc and keep only comments that add information
Functions	`checkout()` (42 lines) and `addBook()` (40 lines) are longer than the rest	Extract the address and payment-method questions into their own methods
Classes	`Main` (about 630 lines) still mixes screen output, input and flow control	Split it by menu (admin, customer, reports) or move checkout logic to a controller class
Objects and Data Structures	Some model classes print to the console (`Payment` , `Notification` , `Publisher.viewBooks()`), mixing domain and presentation	Return the text and let the view print it
Error Handling	`Book.setPrice()` and `Book.updateStock()` silently ignore invalid values; `Lists.find()` returns `null`	Throw an exception, or return `Optional`

Book principle	What we found	Suggested improvement
Smells (magic numbers)	The limits 18, 100 and 8 in `Customer` and the cost 5.0f in `DeliveryDriver.calculateCost()` are written as literals	Replace them with named constants
Law of Demeter	Chains such as `assignment.getDriver().getName()` in `Main`	Add small methods that return the data directly
Unit Tests	The `controller` package has no tests	Add tests for `JsonStorage` and `ReportService` using a temporary folder

Several of these points were detected by our own tests: the defects DEF-BOK-02, DEF-BOK-03 and DEF-CRT-01 are all cases where the error-handling rules of the book are not met yet.

5. Evidence of Refactoring

We applied the book's ideas to the file the user sees first, `Main.java`:

- Length went from 692 to 633 lines (602 to 541 lines of real code).
- Five repeated structures (menus, add-entity, listings, input reading, error display) were replaced by shared helper methods.
- Behavior was not changed: we ran the old and the new version with the same scripted session (more than 200 inputs and 1,219 lines of output) and the console output, the saved JSON files and the CSV reports were identical. This is the book's advice to refactor only with tests or checks that protect the behavior.

6. Conclusion

The project follows most of the book's principles in its structure: clear names, small functions, separated packages, isolated boundaries and a real suite of unit tests. The remaining problems (redundant Javadoc, some console output inside the model, magic numbers and silent error handling in a few methods) are known, documented and easy to fix in the next iteration.

References

Martin, R. C. (2008). *Clean Code: A Handbook of Agile Software Craftsmanship*. Prentice Hall.